

Resume Interview AI — CTS Panel Project Division & Presentation Guide

8-Member Full-Stack Architecture Split (4 Core Sections × 2 Members per Section)

SECTION 1: FRONTEND ARCHITECTURE, UI/UX & VOICE INTEGRATION

MEMBER 1: Frontend Architecture & UI/UX Design Lead — UI Architecture & Component System Engineer

Tech Stack: React 19, Vite, Tailwind CSS, Lucide Icons

Modules: 12 platform views, Sidebar navigation, ScoreRing/Badge/Modal design system, Multi-device responsive layouts.

Speech Script: "I was responsible for the core frontend architecture and design system using React 19 and Tailwind CSS. I built the modular layout, interactive gauges, and responsive navigation across all 12 platform views."

Panel Q: How did you structure the component hierarchy for reusability? **Ans:** "We created atomic components in components/common/ like ScoreRing and Badge, ensuring consistent props, visual tokens, and zero-runtime CSS overhead."

MEMBER 2: Frontend State Management & Voice UI Engineer — Client State & Speech-to-Text Specialist

Tech Stack: React Context API, Web Speech API, Axios Interceptors, sessionStorage

Modules: SessionContext global store, Voice dictation in Mock Interview, Live backend health badge (Online/Offline), 502 error handling.

Speech Script: "I engineered the frontend state management and browser voice capabilities. I built SessionContext with ephemeral sessionStorage, integrated SpeechRecognition for voice dictation, and added live backend health status monitoring."

Panel Q: How does voice dictation work in the browser? **Ans:** "We hook into window.SpeechRecognition with continuous streaming, appending transcribed text tokens directly to the user's answer state in real-time."

SECTION 2: DOCUMENT PROCESSING, NLP & SEMANTIC MATCH ENGINE

MEMBER 3: Resume Ingestion & Document Parsing Lead — Document Parsing & Resume Scoring Engineer

Tech Stack: Python 3.13, PyMuPDF (fitz), Python-docx, Regex Heuristics

Modules: PDF & Word text extraction, Section segmentation (Summary/Skills/Experience/Education), 5-factor quality scoring.

Speech Script: "I built the resume ingestion and parsing pipeline using PyMuPDF and python-docx. I developed algorithms to extract structured text from resumes, segment key sections, and compute an explainable 5-factor resume quality score."

Panel Q: How do you handle multi-column resumes or unstructured formats? **Ans:** "PyMuPDF extracts raw text blocks preserving reading order, followed by multi-pass regex scanning for standard section headers to segment sections reliably."

MEMBER 4: Semantic Matching & Skill Gap Engineer — Job Description Matching & Gap Analysis Specialist

Tech Stack: Scikit-learn (TfidfVectorizer, cosine_similarity), NLP Tokenizer

Modules: Semantic similarity score calculation, Skill classification (Matched, Missing Gaps, Strengths), Resume improvements engine.

Speech Script: "I developed the semantic matching and skill gap engine using Scikit-Learn. I implemented TF-IDF vectorization and Cosine Similarity to calculate match scores and automatically classify technical skills into matched proficiencies, missing gaps, and candidate strengths."

Panel Q: Why choose TF-IDF with Cosine Similarity over large deep learning embeddings? **Ans:** "TF-IDF executes in <50ms with zero API cost/GPU overhead, providing transparent keyword weights so candidates can see exactly which terms influenced their score."

SECTION 3: GROUNDED QUESTION GENERATION & MOCK EVALUATION

MEMBER 5: Grounded Question Generator Lead — Question Generation & Anti-Hallucination Specialist
Tech Stack: FastAPI, Google Gemini API / Grounded Deterministic Engine, Pydantic v2
Modules: Grounded question engine anchored to candidate projects, Filters (Easy/Med/Hard/Expert, Tech/Behavioral), Zero-duplicate tracking.
Speech Script: "I led the Grounded Question Generation engine. I enforced strict algorithmic constraints to ensure every question is anchored to verified candidate projects or target JD requirements, with zero-duplicate history tracking."
Panel Q: How do you prevent the AI from generating generic questions? **Ans:** "Every generation payload passes parsed resume project entities and requires an explicit based_on field linking each question to an identified project or tool."

MEMBER 6: Mock Interview & 6-Axis Evaluation Specialist — Answer Evaluation & STAR Diagnostics Engineer
Tech Stack: Python, STAR Method Heuristics, Domain Concept Dictionary
Modules: 6-Axis evaluation matrix (Accuracy/Completeness/Clarity/etc.), STAR diagnostic analyzer, Senior model answers & follow-up questions.
Speech Script: "I developed the 6-axis answer evaluation engine and mock interview logic. I built algorithms that extract domain concepts, check for technical anti-patterns, evaluate STAR structure, and formulate senior model answers with interactive follow-ups."
Panel Q: How does the system evaluate technical accuracy without a human interviewer? **Ans:** "We match candidate answers against a domain concept dictionary (e.g. B-Trees for SQL, Event Loop for JS) and penalize misconceptions, returning explicit lists of covered vs missed concepts."

SECTION 4: BACKEND INFRASTRUCTURE, DATABASE & REPORTING

MEMBER 7: REST API & Dual-Database Architect — Backend Architecture & Database Engine Specialist
Tech Stack: FastAPI, Motor / PyMongo (MongoDB), AsyncIO, JSON Database Engine
Modules: Asynchronous REST routing, Dual-Database manager with automated MongoDB + Local JSON fallback, Session lifecycle APIs.
Speech Script: "I architected the FastAPI backend and dual-mode database layer. I designed asynchronous REST endpoints and built a database manager that connects to MongoDB or falls back to an async local JSON database if MongoDB is not present."
Panel Q: How does the system handle database operations asynchronously? **Ans:** "We use Motor for MongoDB and asynchronous file I/O for the local database, allowing queries to run non-blockingly on FastAPI's async event loop without thread pool contention."

MEMBER 8: Readiness Analytics, PDF Reports & Automation Lead — Analytics Dashboard, PDF Generation & Deployment Engineer
Tech Stack: ReportLab, Recharts, Cross-Platform Shell Scripts (run.py, run.bat, run.sh)
Modules: Automated ReportLab PDF readiness report export, Readiness analytics charts, 1-click cross-platform launchers for Windows/Mac/Linux.
Speech Script: "I engineered the Readiness Analytics dashboard, the ReportLab PDF export engine, and cross-platform automation. I built the automated PDF generator for evaluation reports and created 1-click execution scripts for Windows, Mac, and Linux environments."
Panel Q: How is the PDF generated on the fly? **Ans:** "We use Python's ReportLab library to compile layout flowables, tables, and styled paragraphs into an in-memory binary stream, returning a downloadable PDF in under 100 milliseconds."

8-Minute CTS Panel Presentation Order (1 Min per Member)

1	Member 1	Introduction & UI Layout System	Dashboard & Responsive Sidebar Navigation
2	Member 2	State Management & Voice Dictation	Live Status Badge & Speech-to-Text Microphone
3	Member 3	Resume Ingestion & 5-Factor Quality Score	Upload Resume & Parsed Score Breakdown
4	Member 4	Semantic Match & Skill Gap Diagnostics	TF-IDF Match % & Missing vs Matched Skills
5	Member 5	Grounded Question Generator & Filters	Generate Filtered Questions Grounded in Resume
6	Member 6	Mock Interview & 6-Axis Evaluation	Submit Answer -> 6-Axis Matrix & Model Answer
7	Member 7	FastAPI Architecture & Dual-Database	Interactive Swagger Docs (/docs) & DB Engine
8	Member 8	Readiness Analytics & PDF Report Export	Analytics Trends & Download PDF Report